Comparison of `==` between Primitives and Objects in Java

In Java, the behavior of the `==` operator differs significantly when comparing **primitive data types** and **objects**. Here's a detailed comparison:

---

1. Comparing Primitives with `==`

When you use `==` to compare **primitive data types**, it **compares the actual values** stored in the variables.

- **Primitive Data Types:** These are basic data types like `int`, `char`, `float`, `double`, `boolean`, etc.
- **Behavior:** When comparing two primitive values, `==` checks if the values are exactly the same.

Example:

```
public class PrimitiveComparison {
    public static void main(String[] args) {
        int num1 = 10;
        int num2 = 10;
        int num3 = 20;

        System.out.println("num1 == num2: " + (num1 == num2)); // true,
because 10 == 10
        System.out.println("num1 == num3: " + (num1 == num3)); // false,
because 10 != 20
    }
}
```

- **Output:**

```
num1 == num2: true
num1 == num3: false
```

In this case, since both `num1` and `num2` hold the same primitive value (`10`), `==` returns `true` for `num1 == num2`. For `num1 == num3`, since the values are different (`10` and `20`), it returns `false`.

2. Comparing Objects with `==`

When you use `==` to compare **objects**, it **compares the memory addresses** (references) of the objects, not their contents. This means that `==` checks whether both references point to the **same object in memory**.

- **Objects:** In Java, objects are instances of classes (e.g., `String`, `ArrayList`, or any user-defined class).
- **Behavior:** When comparing two object references with `==`, it checks if both references point to the same location in memory (i.e., if they are the same object).

Example:

```
public class ObjectComparison {
    public static void main(String[] args) {
        String str1 = new String("Hello");
        String str2 = new String("Hello");
        String str3 = str1;

        System.out.println("str1 == str2: " + (str1 == str2)); // false,
because they are different objects
        System.out.println("str1 == str3: " + (str1 == str3)); // true,
because both reference the same object
    }
}
```

- **Output:**

```
str1 == str2: false
str1 == str3: true
```

In this example:

- `str1` and `str2` contain the same value ("Hello"), but since they are two different objects (created using the `new` keyword), `==` returns `false`.
- `str3` is assigned the reference of `str1`, so `str1 == str3` returns `true` because both variables refer to the **same object** in memory.

Key Differences between `==` for Primitives and Objects

Aspect	Primitive Data Types	Objects
What is compared	The **actual value** of the variable.	The **memory address** (reference) of the object.
Behavior with `==`	Compares the **value** stored in the variable.	Compares the **reference** (memory address) of the objects.
Example	`10 == 10` returns `true`.	`new String("Hello") == new String("Hello")` returns `false`.
Value equality	Direct comparison of values.	Checks if both references point to the same object.
Primitives Example	`int a = 5; int b = 5; a == b;`	Not applicable.
Objects Example	Not applicable.	`String s1 = new String("A"); String s2 = new String("A"); s1 == s2;` returns `false`.

Important Consideration:

For comparing the **contents** of objects (not their references), Java provides the `.equals()` method, which is overridden in many classes (like `String`) to compare the actual contents of the objects.

Example:

```
public class EqualsComparison {
    public static void main(String[] args) {
        String str1 = new String("Hello");
        String str2 = new String("Hello");

        // Comparing contents using equals() method
        System.out.println("str1.equals(str2): " + str1.equals(str2)); //
true, because the contents are the same
    }
}
```

- **Output:**

```
str1.equals(str2): true
```

Here, `equals()` compares the **value/content** of the `String` objects (`"Hello"` in both cases), so it returns `true`.

Conclusion

- **Primitives:** The `==` operator compares the actual values of primitive data types.
- **Objects:** The `==` operator compares references (memory addresses) of objects, not their contents. To compare the actual contents, you should use the `.equals()` method.